Topic 11: Generics and Error Handling - Complete Code

Task 1: A Generic Stack

```
struct Stack<Element> {
    private var items: [Element] = []

    var isEmpty: Bool {
        items.isEmpty
    }

    var count: Int {
        items.count
    }

    var top: Element? {
        items.last
    }

    mutating func push(_ item: Element) {
        items.append(item)
    }

    @discardableResult
    mutating func pop() -> Element? {
        items.popLast()
    }
}

func largest<T: Comparable>(in items: [T]) -> T? {
    guard var best = items.first else { return nil }
    for item in items.dropFirst() where item > best {
        best = item
    }
    return best
}

func swapValues<T>(_ a: inout T, _ b: inout T) {
    let temp = a
    a = b
    b = temp
}

struct Student: Comparable {
    let name: String
    let marks: Int
```

```
    static func < (lhs: Student, rhs: Student) -> Bool {
        lhs.marks < rhs.marks
    }
}

var intStack = Stack<Int>()
intStack.push(10)
intStack.push(20)
intStack.push(30)
print("Int stack top: \(intStack.top ?? 0), count: \(intStack.count)")
intStack.pop()
print("After pop, top: \(intStack.top ?? 0), count: \(intStack.count)")

var stringStack = Stack<String>()
stringStack.push("Swift")
stringStack.push("Generics")
print("String stack top: \(stringStack.top ?? "none"), empty: \(stringStack.isEmpty)")

while let word = stringStack.pop() {
    print("Popped \(word)")
}
print("String stack empty now: \(stringStack.isEmpty)")

if let biggest = largest(in: [4, 17, 9, 2]) {
    print("Largest number: \(biggest)")
}

if let lastWord = largest(in: ["apple", "mango", "banana"]) {
    print("Largest string: \(lastWord)")
}

let emptyNumbers: [Int] = []
print("Largest of empty array: \(String(describing: largest(in: emptyNumbers)))")

let students = [
    Student(name: "Asha", marks: 82),
    Student(name: "Ravi", marks: 91),
    Student(name: "Meena", marks: 76)
]
if let topper = largest(in: students) {
    print("Topper: \(topper.name) with \(topper.marks)")
}

var first = 1
var second = 2
swapValues(&first, &second)
print("Numbers after swap: \(first), \(second)")
```

```
var city = "Chennai"
var town = "Madurai"
swapValues(&city, &town)
print("Strings after swap: \(city), \(town)")
```

Output

```
Int stack top: 30, count: 3
After pop, top: 20, count: 2
String stack top: Generics, empty: false
Popped Generics
Popped Swift
String stack empty now: true
Largest number: 17
Largest string: mango
Largest of empty array: nil
Topper: Ravi with 91
Numbers after swap: 2, 1
Strings after swap: Madurai, Chennai
```

Task 2: ATM Withdrawal with Errors

```
enum ATMError: Error {
    case insufficientFunds(shortBy: Double)
    case invalidAmount
    case dailyLimitExceeded
}

struct Account {
    static let dailyLimit = 20_000.0

    private(set) var balance: Double

    mutating func withdraw(_ amount: Double) throws {
        guard amount > 0 else {
            throw ATMError.invalidAmount
        }
        guard amount <= Account.dailyLimit else {
            throw ATMError.dailyLimitExceeded
        }
        guard amount <= balance else {
            throw ATMError.insufficientFunds(shortBy: amount - balance)
        }
        balance -= amount
    }
}
```

```
func attempt(_ amount: Double, on account: inout Account) {
    do {
        try account.withdraw(amount)
        print("Withdrew \(amount). New balance: \(account.balance)")
    } catch ATMError.insufficientFunds(let shortBy) {
        print("Insufficient funds. Short by \(shortBy)")
    } catch ATMError.invalidAmount {
        print("Invalid amount: \(amount)")
    } catch ATMError.dailyLimitExceeded {
        print("Daily limit of \(Account.dailyLimit) exceeded")
    } catch {
        print("Unexpected error: \(error)")
    }
}

var account = Account(balance: 10_000)

for amount in [500, -10, 25_000, 15_000] as [Double] {
    attempt(amount, on: &account)
}

let worked = (try? account.withdraw(100)) != nil
print("Quick withdrawal worked: \(worked)")

let failed = (try? account.withdraw(50_000)) != nil
print("Huge withdrawal worked: \(failed)")
print("Final balance: \(account.balance)")
```

Output

```
Withdrew 500.0. New balance: 9500.0
Invalid amount: -10.0
Daily limit of 20000.0 exceeded
Insufficient funds. Short by 5500.0
Quick withdrawal worked: true
Huge withdrawal worked: false
Final balance: 9400.0
```

Task 3: Result Type and defer

```
import Foundation

enum AgeError: Error {
   case empty
   case notANumber
   case outOfRange
}

enum ReportError: Error {
   case writeFailed
}

func parseAge(_ text: String) throws -> Int {
   let trimmed = text.trimmingCharacters(in: .whitespaces)
   guard !trimmed.isEmpty else {
      throw AgeError.empty
   }
   guard let age = Int(trimmed) else {
      throw AgeError.notANumber
   }
   guard (1...120).contains(age) else {
      throw AgeError.outOfRange
   }
   return age
}

func describeAge(_ text: String) {
   let result = Result { try parseAge(text) }

   switch result {
   case .success(let age):
      print("'\(text)' is a valid age: \(age)")
   case .failure(let error):
      print("'\(text)' is invalid: \(error)")
   }
}

for input in ["25", "", "abc", "150"] {
   describeAge(input)
}

func saveReport(shouldFail: Bool = false) throws {
   print("Opening file")
   defer {
      print("Closing file")
```

```
    }

    if shouldFail {
        throw ReportError.writeFailed
    }
    print("Report saved")
}

do {
    try saveReport()
} catch {
    print("Caught: \(error)")
}

print("-----")

do {
    try saveReport(shouldFail: true)
} catch {
    print("Caught: \(error)")
}
```

Output

```
'25' is a valid age: 25
'' is invalid: empty
'abc' is invalid: notANumber
'150' is invalid: outOfRange
Opening file
Report saved
Closing file
-----
Opening file
Closing file
Caught: writeFailed
```